

Data lake : outils, briques et facture

1. Présentation des technologies

Pandas

Pandas **sert à** manipuler des données en mémoire sous forme de DataFrames.

Avantages : très simple, très flexible, idéal pour l'exploration et les petits jeux de données. **Inconvénients** : limité par la RAM, lent sur les gros volumes.

Limite : au-delà de quelques millions de lignes, Pandas n'est plus adapté.

DuckDB

DuckDB **est** un moteur SQL en mémoire optimisé pour l'analytique.

Avantages : extrêmement rapide, lit les fichiers Parquet directement, fonctionne sans serveur, parfait pour la volumétrie SIRENE.

Inconvénients : pas conçu pour le multi-utilisateur ou les workloads transactionnels.

Limite : ne remplace pas une base de production (PostgreSQL, Snowflake...).

PostgreSQL

PostgreSQL **est** : une base de données relationnelle robuste, fiable et transactionnelle.

Avantages : stable, SQL complet, indexation, transactions, idéal pour stocker l'entrepôt final.

Inconvénients : moins performant que DuckDB pour les requêtes analytiques massives.

Limite : sur des dizaines de millions de lignes, nécessite optimisation ou architecture distribuée.

dbt

dbt **est** : un outil de transformation SQL orienté Data Warehouse.

Avantages : versionning, documentation automatique, tests, lineage, standardisation des modèles.

Inconvénients : nécessite un Data Warehouse SQL (Snowflake, BigQuery, Postgres).

Limite : ne gère pas l'ingestion ni les pipelines complexes (orchestration externe nécessaire).

DuckDB + httpfs

Permet de lire des fichiers Parquet distants via HTTP.

Avantages : lecture directe depuis data.gouv.fr, pas besoin de télécharger les fichiers.

Inconvénients : dépend de la stabilité du réseau.

Limite : pas adapté si les fichiers dépassent plusieurs dizaines de Go.

Outils	À quoi ça sert	Les avantages	Les inconvénients	Ses limites
Pandas	Manipulation de données en mémoire	Simple, flexible, idéal pour l'exploration	Lent sur gros volumes, dépend de la RAM	Pas adapté > 5–10 M de lignes
DuckDB	Moteur SQL analytique en mémoire	Ultra-rapide, lit Parquet, parfait pour SIRENE	Pas multi-utilisateur, pas transactionnel	Ne remplace pas une base de production
PostgresSQL	Base relationnelle pour stockage durable	Fiable, robuste, SQL complet	Moins performant pour l'analytique massive	Peut saturer sur > 50 M de lignes
dbt	Transformations SQL, tests, documentation	Lineage, tests, standardisation	Nécessite un Data Warehouse	Ne fait pas l'ingestion
DuckDB + httpfs	Lecture de Parquet via HTTP	Pas de téléchargement, très rapide	Dépend du réseau	Pas adapté aux fichiers géants

Partie 2 – Veille sur les briques du data lake

1. Stockage objet (modèle S3)

Simple Storage Service (S3) est un service de stockage cloud évolutif conçu pour garder des fichiers (parquet, csv, images...) dans des “**buckets**”, comme de grands dossiers dans le cloud. On peut accéder aux fichiers avec une simple URL. S3 est très utilisé dans les data lakes car il permet de stocker n’importe quel volume de données, avec une bonne sécurité, une forte durabilité et un coût assez bas. **Sa limite** : on ne peut pas faire des requêtes SQL directement dessus, il faut un moteur comme DuckDB ou Spark.

- **Référence:**« A. Glue, [Overview of Amazon Web Services - Overview of Amazon Web Services](#) »

2. Formats colonnes (Parquet vs CSV)

- **Parquet** est un format de stockage en colonnes. Il permet de lire les données très vite, de réduire la taille des fichiers grâce à la compression, et d’améliorer les requêtes. Selon DuckDB v2.0 Preview, le moteur Parquet bénéficie d’optimisations majeures : I/O asynchrone, pruning avancé, partition awareness et un nouveau format de stockage plus rapide (DuckDB Project, 2026).

Parquet lit les données tabulaires (nombres, textes, dates). La documentation Apache Parquet précise qu’il n’est pas adapté aux fichiers non tabulaires comme les images, vidéos, PDF ou audio (Apache Software Foundation)

- **Référence** : <https://parquet.apache.org/documentation/latest/> ([parquet.apache.org in Bing](#)) - [A Preview of DuckDB v2.0 – DuckDB](#) -Highlights.” DuckDB.org, Aug. 2026.

- **CSV**

CSV(Comma-Separated Values):stocke des données tabulaires : chaque ligne est un enregistrement et chaque colonne est séparée par une virgule. Selon la documentation du W3C, c’est un format simple et universel, compatible avec Excel ou Google Sheets (W3C, 2015). Sur Hacker News, plusieurs ingénieurs expliquent utiliser DuckDB pour lire des CSV locaux très rapidement, mais précisent que le format CSV est non compressé et devient lent sur de très gros volumes (Hacker News, 2026).

- **Référence** : [Model for Tabular Data and Metadata on the Web](#)
[A Preview of DuckDB v2.0 | Hacker News](#) - (Hacker News, 2026)

3. Bibliothèques pour manipuler des données tabulaires : Polars et Pandas

- **Pandas** est une bibliothèque pour manipuler et analyser des données sous forme de DataFrames en mémoire. Selon la documentation officielle, Pandas charge les données en RAM, ce qui limite son utilisation sur des très gros volumes (Pandas Development Team). Le blog *Dagster, dbt, duckdb as new local MDS* explique que Pandas est souvent remplacé par DuckDB ou Polars dans les data stacks modernes, car ces moteurs sont plus rapides pour les gros volumes (Milicevic & Heiler, 2023).
- **Référence** : [pandas documentation — pandas 3.0.5 documentation](#)
- [Dagster, dbt, duckdb as new local MDS | Georg Heiler](#)
- **Polars** est une bibliothèque très rapide, écrite en Rust, qui manipule les données en colonnes. La documentation officielle décrit Polars comme un moteur colonne performant, avec un mode **Lazy** qui optimise les calculs avant exécution (Polars Contributors) ce qui réduit le temps total.. Le blog *Dagster, dbt, duckdb as new local MDS* cite Polars comme un moteur moderne performant utilisé pour remplacer Pandas dans les gros volumes (Milicevic & Heiler, 2023).
- **Sa limite** : elle est encore moins connue et certaines fonctions avancées sont moins complètes que Pandas.

- **Référence** : [Polars — DataFrames for the new era](#)
- [Dagster, dbt, duckdb as new local MDS | Georg Heiler](#)

4. Moteur analytique sur fichiers (DuckDB) vs moteur distribué (Spark)

- **DuckDB : moteur analytique sur fichiers :**

DuckDB est un moteur SQL qui lit directement les fichiers Parquet. Il sert à analyser des données sur une seule machine, très rapidement. Selon la documentation officielle, DuckDB est un moteur analytique local optimisé pour les formats colonnes comme Parquet (DuckDB Authors). La page “A Preview of DuckDB v2.0” montre que DuckDB améliore fortement la lecture des fichiers grâce à l’I/O asynchrone, au pruning et à un nouveau format de stockage, tout en restant un moteur non distribué. Le blog “Dagster, dbt, duckdb as new local MDS” explique que DuckDB est idéal pour les notebooks, les projets locaux et les data lakes légers, mais non adapté aux clusters.

- **Références** : [Documentation – DuckDB](#)
- [A Preview of DuckDB v2.0 – DuckDB](#)
- [Dagster, dbt, duckdb as new local MDS | Georg Heiler](#)

- **Apache Spark : moteur distribué pour le Big Data :**

Apache Spark est une plateforme open-source pour traiter des très grandes données (Big Data). Il sert à analyser des données à grande échelle, sur plusieurs machines en parallèle. Selon la documentation officielle, Spark propose du batch, du streaming, du SQL, du machine learning et du traitement de graphes (Apache Software Foundation). Spark est utilisé quand les données sont trop grosses pour un seul ordinateur.

- **Capacités de Spark : Spark peut faire :**

- **du batch** (gros fichiers, comme SIRENE France entière),
- **du streaming** (données en temps réel),
- **du SQL**,
- **du machine learning**,
- **du traitement de graphes**. Ces fonctions sont décrites dans la documentation Spark 4.2.0.

- **Architecture distribuée** : Spark utilise une architecture distribuée composée de :

- **Driver** : organise le travail et crée le plan des opérations (DAG).
- **Cluster Manager** : gère les ressources (CPU, RAM, machines).
- **Workers** : machines du cluster qui exécutent les calculs.
- **Executors** : programmes sur les Workers qui font les calculs en mémoire.
- **RDD** : données réparties sur plusieurs machines, immuables et parallélisées.

Cette architecture est décrite dans *Cluster Mode Overview – Spark Documentation*.

- **Pourquoi Spark est rapide :**

- il travaille **en mémoire (RAM)**,
- il exécute les calculs **en parallèle** sur plusieurs machines,
- il évite d’écrire sur le disque à chaque étape.

L’article scientifique *Apache Spark: A Big Data Processing Engine* (IEEE) confirme que Spark est conçu pour les volumes que ne peut pas gérer un seul ordinateur.

- **Limites** : Spark est puissant, mais :

- il est **complexe à installer**,
- il demande une **infrastructure de cluster** (Kubernetes, YARN),
- il n’est **pas utile pour les petits projets locaux**.

Le blog Databricks Spark 3.0 explique que Spark est idéal pour les clusters, mais trop lourd pour un usage local simple.

- **En résumé** Spark = distribué, puissant, idéal pour le Big Data et les clusters.

- **Références** : [Overview - Spark 4.2.0 Documentation](#)

- [Cluster Mode Overview - Spark 4.2.0 Documentation](#)
- [Apache Spark: A Big Data Processing Engine | IEEE Conference Publication | IEEE Xplore](#)
- [Présentation d'Apache Spark 3.0 - Le blog Databricks](#)

5. Base orientée documents (MongoDB) vs relationnelle (PostgreSQL)

• MongoDB : base orientée documents

MongoDB est une base NoSQL orientée documents. Elle stocke les données sous forme de documents JSON/BSON, ce qui offre une structure flexible pour gérer des données variées. Selon la documentation officielle, MongoDB est rapide, scalable, et supporte de très gros volumes grâce aux index, à l'agrégation et à la réplication. MongoDB s'intègre facilement au cloud via MongoDB Atlas. **Sa limite** : il est moins adapté aux relations complexes ou aux modèles structurés (modèle étoile, SCD2, entrepôt de données), où une base relationnelle est plus efficace.

- **Références** : [What is MongoDB? - Database Manual - MongoDB Docs](#)
- [MongoDB Atlas: Multi-Cloud Database Service - Atlas - MongoDB Docs](#)
- [BSON \(Binary JSON\) Serialization](#)

• PostgreSQL : base relationnelle

PostgreSQL est une base de données relationnelle très utilisée dans les projets professionnels. Selon la documentation officielle, les données sont organisées en tables, avec des colonnes, des clés et des relations, ce qui en fait une solution idéale pour les modèles structurés comme un entrepôt de données, un modèle étoile ou un SCD2 (PostgreSQL Global Development Group).

PostgreSQL offre des transactions fiables (ACID), des index puissants comme B-Tree, GIN et GiST, des vues matérialisées, des fonctions SQL avancées, et des extensions comme PostGIS pour la géographie (PostGIS Project). Il garantit une forte cohérence des données et une sécurité solide, ce qui le rend très performant pour les requêtes complexes, les jointures et les modèles analytiques.

- **Sa limite** : PostgreSQL est moins flexible que MongoDB pour des données très variées ou non structurées, et peut devenir lent si les données sont énormes et mal indexées.
- **Références** : [PostgreSQL: Documentation](#)
- [PostgreSQL: Documentation: 18: Chapter 11. Indexes](#)
- [PostgreSQL: Documentation: 18: CREATE MATERIALIZED VIEW](#)
- [Documentation | PostGIS](#)

6. Conteneurisation (Docker)

Docker permet de mettre une application dans un **conteneur**, c'est-à-dire une petite boîte qui contient tout ce qu'il faut pour l'exécuter (code, dépendances, configuration). Selon la documentation officielle, les conteneurs assurent un **environnement identique** partout, ce qui rend les projets **reproductibles** et faciles à déployer (Docker Documentation). Docker est très utilisé pour simplifier le développement, le test et le déploiement dans le cloud ou dans un pipeline de données.

- **Références** : [What is Docker? | Docker Docs](#)
- [Docker Engine | Docker Docs](#)

Partie 3 : une question ouverte à partir du brief d'industrialisation :

1. Stockage objet S3

Avec des fichiers très lourds, le stockage local devient lent et difficile à partager. Amazon S3 permet de garder les fichiers volumineux sans saturer la machine, d'y accéder via URL, de les utiliser sur plusieurs postes et d'éviter les problèmes Git liés aux gros fichiers.

Selon le whitepaper AWS, S3 est conçu pour la scalabilité et les très gros volumes (Glue, *AWS Overview*).

⇒ **Impact positif** : gestion plus simple des gros fichiers + collaboration + exécution distribuée.

Référence :

- [Overview of Amazon Web Services - Overview of Amazon Web Services](#)

2. Formats colonnes (Parquet) pour réduire le temps d'exécution

Les fichiers CSV France entière sont lourds et lents à lire. Le format Parquet permet :

- de réduire **la taille** des fichiers,
- de lire seulement **les colonnes** nécessaires,
- d'**accélérer** fortement les requêtes analytiques.

La documentation Apache Parquet confirme que le format est optimisé pour les données **tabulaires** et la compression. DuckDB v2.0 ajoute des optimisations comme l'I/O asynchrone et le pruning, ce qui réduit encore le temps d'exécution.

⇒ **Impact positif** : temps d'exécution plus court + moins de RAM utilisée.

Références :

- <https://parquet.apache.org/documentation/latest/>
- <https://duckdb.org/2026/08/17/duckdb-20-highlights>

3. Polars ou DuckDB pour remplacer Pandas

Pandas charge tout en mémoire, ce qui pose problème sur des fichiers France entière. Des moteurs plus modernes comme **Polars** ou **DuckDB** permettent :

- de lire Parquet très rapidement,
- de travailler en mode Lazy (Polars),
- d'exécuter des requêtes SQL optimisées (DuckDB),
- de réduire la consommation mémoire.

La documentation Polars décrit un moteur colonne très performant (Polars Contributors). Le blog Dagster/dbt/DuckDB explique que Polars et DuckDB remplacent Pandas dans les data stacks modernes (Milicevic & Heiler, 2023). ⇒ **Impact positif** : calculs plus rapides + moins de RAM + pipelines plus stables.

Références :

- <https://pola.rs>
- <https://georgheiler.com/2023/12/11/dagster-dbt-duckdb-as-new-local-mds>

4. Spark pour exécuter sur plusieurs machines

Sur la France entière, certains traitements deviennent trop lourds pour une seule machine. Un moteur distribué comme **Apache Spark** permettrait :

- de répartir les calculs sur plusieurs machines,
- de traiter des volumes massifs,
- d'exécuter du batch, du streaming, du SQL et du ML.

La documentation Spark décrit l'architecture Driver / Cluster Manager / Workers / Executors. L'article IEEE confirme que Spark est conçu pour les volumes qu'un seul ordinateur ne peut pas gérer (Shaikh et al., 2019). ⇒ **Impact positif** : scalabilité + réduction du temps d'exécution + robustesse.

Références :

- <https://spark.apache.org/docs/latest/>
- <https://spark.apache.org/docs/latest/cluster-overview.html>
- <https://ieeexplore.ieee.org/document/8988541>

5. PostgreSQL pour structurer les données France entière

Les fichiers CSV France entière sont difficiles à manipuler, à versionner et à requêter. Une base relationnelle comme **PostgreSQL** permettrait :

- de structurer les données,
- de faire des jointures complexes,
- d'utiliser des index puissants,
- de créer des vues matérialisées pour accélérer les analyses.

La documentation PostgreSQL confirme ces capacités .

⇒ **Impact positif** : requêtes plus rapides + meilleure organisation + analyses plus fiables.

Références :

- <https://www.postgresql.org/docs/>
- [PostgreSQL: Documentation: 18: Chapter 11. Indexes](#)
- [PostgreSQL: Documentation: 18: CREATE MATERIALIZED VIEW](#)

6. Docker pour stabiliser l'environnement d'exécution

Lors du projet, les environnements locaux peuvent varier (versions Python, dépendances, OS). Docker permet :

- d'avoir un environnement identique pour tout le monde,
- de déployer facilement sur un serveur ou un cluster,
- d'éviter les erreurs "ça marche chez moi mais pas chez toi".

La documentation Docker explique que les conteneurs assurent un environnement reproductible (Docker Documentation). ➔ **Impact positif** : moins d'erreurs + déploiement plus simple + industrialisation plus propre.

Références :

- [What is Docker? | Docker Docs](#)
- [Docker Engine | Docker Docs](#)

Conclusion générale

Les briques les plus utiles pour industrialiser un traitement France entière sont :

- **S3** pour stocker les gros fichiers,
- **Parquet** pour accélérer les lectures,
- **Polars** / **DuckDB** pour réduire la RAM,
- **Spark** pour exécuter sur plusieurs machines,
- **PostgreSQL** pour structurer les données,
- **Docker** pour stabiliser l'environnement.

Elles répondent directement aux difficultés rencontrées : lenteur, mémoire saturée, complexité Git, et limites d'une machine unique.

Partie 4 : estimation du coût d'hébergement :

(voir le tableau de comparaison en bas)

* Pour ce brief, je n'ai pas à héberger tout le projet (pipeline, scripts, API, interface, orchestrateur).

La Partie 4 demande uniquement d'estimer le coût d'hébergement de l'observatoire, c'est-à-dire de la base de données PostgreSQL de 22 Go. C'est pour cela que je choisis une machine adaptée au stockage, à la RAM et au SLA nécessaires pour la base, et non une machine plus grande pour héberger l'ensemble du projet. (je n'ai pas construit un pipeline cloud-ready /j'ai pas de conteneurs prêts à déployer)

1. OVHcloud (France) :

[Tarif Cloud : comparatif des offres Public Cloud | OVHcloud France](#)

A) Machine de production :

(Ma base de données fait 22 Go → besoin d'une machine "Balanced" avec ~15 Go RAM)

RAM $\geq 25\%$ de la taille de la base → $22\text{ Go} \times 0.25 = 5.5\text{ Go}$

RAM $\geq 2 \times$ taille des index → souvent 4-8 Go

+ marge pour requêtes analytiques → +4 Go

Total recommandé : 12-16 Go RAM

Comment choisir le CPU ?

Le type de charge: mon observatoire :

- fait des requêtes API
- stocke des données
- fait quelques agrégations
- n'a pas de calcul intensif
- n'a pas de machine learning
- n'a pas de streaming

⇒ Donc **4 vCPU** est largement suffisant.

- **Instance :** b2-15
- **Specs :** 4 vCPU, 15 Go RAM
- **Prix à l'heure :** 0,1342 € HT / heure (prix Savings Plans)
- **Prix mensuel (730 h) :** ~ 98 € / mois
- **SLA :** 99,99 % (Gamme **Balanced**)

[\(Virtual Machine \(VM\) Instances - instance cloud computing | OVHcloud France \)](#)

B) Machine de Développement

- **Instance :** t2-3
- **Specs :** 2 vCPU, 3 Go RAM
- **Prix à l'heure :** 0,029 € HT / heure
- **Prix mensuel (730 h) :** ~ 21 € / mois
- **SLA :** 99,90 % (Gamme **Discovery**)

2. Scaleway (France)

(alternative française fiable, prix compétitifs, bonne disponibilité)

A) Machine de production

General Purpose = équilibre prix/performance, adapté à la production

- **Instance :** General Purpose (PLAY2-MICRO)

- **Specs** : 4 vCPU, 8 Go RAM
- **Prix mensuel** : €0.05508 / heure → ~ **40,20 € / mois**
- **SLA** : **99,99 %** (General Purpose = production)

* (PLAY2-MEDIUM → 16 Go (mais **beaucoup plus cher**, pas dans la gamme cost-effective))

En pratique : Une base PostgreSQL de 22 Go **peut tourner avec 8 Go RAM**

[Virtual Instances Pricing | Scaleway](#)

Pourquoi Catégorie **General Purpose** ?

- serveurs stables pour backend et bases de données.
- 8 Go RAM suffisent pour une base de 22 Go si les index sont optimisés.
- SLA disponible (confirmé dans la FAQ : “Yes, we offer Service Level Agreements”)

B) Machine de développement

(besoin d’une machine légère pour tests, catégorie “Development”)

[Virtual Instances Pricing | Scaleway](#)

- **Instance** : DEV1-M (Development)
- **Specs** : 3 vCPU, 4 GB RAM
- **Prix** : **€0.0202 / heure** → ~ **€14.74 / mois**
- **SLA** : 99,90 %
- **Pourquoi DEV1-M ?**
 - Conçu pour le développement (dev/test/CI/CD).
 - 4 Go RAM suffisent pour tester mon pipeline sans charger les 22 Go.
 - Prix raisonnable.

Option économique : DEV1-S (Development)

- **Specs** : 2 vCPUs, 2 GB RAM
- **Prix** : €0.00898 / heure → ~ **€6.55 / mois**
- Suffisant pour des tests légers.

3. AWS (USA)

(AWS = leader mondial, très haute disponibilité, adapté aux bases de données)

A) Machine de production

La machine adaptée à une base PostgreSQL de 22 Go est :

Instance : t3.large

Specs : 2 vCPU, 8 Go RAM

Prix : ≈ 63,4 € / mois

⇒ (0,0944 × 730 = 68,912 USD/mois ⇒ × 0,92 = 63,39 € / mois)

SLA : **99,99 %** (SLA EC2 standard)

[Tarification d'instance à la demande EC2](#)

B) Machine de Développement [Amazon EC2 T3 Instances – Amazon Web Services \(AWS\)](#)

- **Instance** : t3.micro
- **Specs** : 2 vCPU, 1 Go RAM
- **Prix** : ~ **3,5€/mois** \$0.0052/H (\$0,003/H-year)
(0,0052 × 730 = 3,796 USD/mois ⇒ × 0,92 = 3,49 € / mois)
- **SLA** : **99,50 %**

4. Hetzner

[Cloud-hosting provider for developers & teams](#)

Shared Resources → dev/test (sections [sec8]–[sec11])

Dedicated Resources / General Purpose → production (sections [sec12]–[sec13])

SLA garanti : **99,9 % uptime** (section [sec49])

A) Machine de production (Allemagne) (Not Available en France)

[Cheap cloud-hosting – low vps prices for developers](#)

Pour une base PostgreSQL de 22 Go, il faut une machine avec 16 Go RAM minimum.
La machine adaptée esy :

- Instance : CX43 (Cost-Optimized)
- Specs : **8 vCPU, 16 GB RAM**
- Prix : **€16.49 / mois** (Prix extrêmement bas pour cette puissance)
- SLA : 99.9 % (**la SLA officielle Hetzner Cloud.**)
- Zones : FSN / NBG / HEL

B) Machine de Développement

- Instance : CX23
- Specs : **2 vCPU, 4 GB RAM**
- Prix : **€5.99 / mois**
- SLA : **99,9 %**
- Zones : FSN / NBG / HEL

(4 Go RAM = parfait pour tester ton pipeline. / Prix ultra bas. / Conçu pour dev/test)

Hébergeur	Profil	Instance	Specs	Prix mensuel	SLA	Région
OVHcloud (FR)	Production	b2-15	4 vCPU, 15 Go RAM	98 €	99,99 %	France
OVHcloud (FR)	Développement	t2-3	2 vCPU, 3 Go RAM	21 €	99,90 %	France
Scaleway (FR)	Production	PLAY2-MICRO (General Purpose)	4 vCPU, 8 Go RAM	40,20 €	99,99 %	France
Scaleway (FR)	Développement	DEV1-M	3 vCPU, 4 Go RAM	14,74 €	99,90 %	France
AWS (USA)	Production	t3.large	2 vCPU, 8 Go RAM	63,4 €	99,99 %	France
AWS (USA)	Développement	t3.micro	2 vCPU, 1 Go RAM	3,5 €	99,50 %	France
Hetzner (DE)	Production	CX43 (Cost-Optimized)	8 vCPU, 16 Go RAM	16,49 €	99,99 %	FSN / NBG / HEL
Hetzner (DE)	Développement	CX23	2 vCPU, 4 Go RAM	5,99 €	99,99 %	FSN / NBG / HEL

⇒ **Pour héberger mon observatoire en production,**

Au début : Je choisis **Hetzner CX43** pour mon observatoire : 16 Go RAM, SLA 99,9 %, prix 16,49 €/mois — c'est le meilleur compromis entre performance, fiabilité et coût .

Hetzner Pas disponible en France :

⇒ Je choisis **OVHcloud** pour mon observatoire : c'est français, conforme à ma recommandation RAM (15 Go), SLA 99,99 %, et adapté à la production. Scaleway est moins cher mais insuffisant en RAM, et Hetzner/AWS ne sont pas en France.

A. coût mensuel total en production :

OVHcloud : 98 €

Scaleway : 40,20 €

AWS : 63,4 €

Hetzner : 16,49 €

Moyenne : ~ 54,2 € / mois

⇒ **Mon observatoire coûterait** environ 55–60 €/mois en production.

B. Écart entre développement et production

OVHcloud : 98 € vs 21 €

Scaleway : 40,20 € vs 14,74 €

AWS : 63,4 € vs 3,5 €

Hetzner : 16,49 € vs 5,99 €

Moyenne dev : ~ 11,8 € / mois

Moyenne prod : ~ 54,52 € / mois

⇒ **L'écart moyen est d'environ 43 €/mois.**

les hypothèses de variation des prix :

1. Les prix varient selon la région

- OVHcloud : France
- Scaleway : France
- AWS : France
- Hetzner : Allemagne / Finlande (pas de France)

2. Le type d'instance (gamme) :

Les gammes “**production**” ont un **SLA** plus **élevé** → donc **plus chères**.

3. Les ressources (vCPU, RAM)

Plus la **RAM** est **élevée**, plus le **prix augmente** (surtout pour les bases de données).

4. Le stockage :

Le stockage (disque) n'est pas inclus dans ces estimations. Il ajoute généralement **5 à 10 €/mois** selon l'hébergeur. (**Le coût dépend aussi de la taille (Go) et du type de disque.**)

5. le trafic sortant

Non inclus dans mes calculs. Le trafic sortant est souvent facturé :

- au **Go** (AWS, Scaleway, Hetzner)
- ou inclus dans un quota (OVHcloud)

⇒ Pour un observatoire (API + dashboard), le trafic est faible → coût marginal.

6. Le mode de facturation

Les prix varient selon :

- **à l'heure** (On-Demand)
- **au mois** (forfait)

- **Savings Plans / Engagement 1 an ou 3 ans**
- **Burstable vs Dedicated CPU**

⇒ J'ai choisi **On-Demand / prix mensuel standard** pour simplifier.

7. Le SLA (Service Level Agreement)

Un SLA élevé (99,99 %) coûte plus cher qu'un SLA standard (99,9 %).

⇒ Les machines de production ont un SLA plus élevé → donc plus chères.

Résumé :

Les prix varient selon **la région, la gamme d'instance (production vs développement), les ressources (vCPU/RAM), le stockage, le trafic sortant, le mode de facturation (On-Demand vs engagement), et le SLA.**

Pour simplifier, j'ai choisi les prix On-Demand, et je n'ai pas inclus le stockage ni le trafic sortant (coût marginal pour un observatoire).